

24.RKNN-ToolkitLite2基本使用

1. 基本介绍

RKNN-Toolkit Lite2为用户提供板端模型推理的Python接口，方便用户使用Python语言进行AI应用开发。

注意：在使用RKNN-Toolkit Lite2开发AI应用前，需要通过RKNN-Toolkit2将各深度学习框架导出的模型转成RKNN模型。

2. 搭建环境

建议版本：RK3588: Debian: 11 (aarch64) py3.9、ubuntu20(aarch64)py3.8

1.开发板安装miniconda，然后创建conda环境

将Miniconda3-latest-Linux-aarch64.sh 安装包拷贝到开发板上

```
root@topeet:~# ls
Miniconda3-latest-Linux-aarch64.sh
root@topeet:~# chmod +x Miniconda3-latest-Linux-aarch64.sh
root@topeet:~# ls
Miniconda3-latest-Linux-aarch64.sh
root@topeet:~#
```

然后使用“./Miniconda3-latest-Linux-aarch64.sh”安装Miniconda，输入回车和“yes”，等待安装完成，安装完成如下图所示：

然后使用“source ~/.bashrc”命令更新环境变量，可以看到在终端前会出现（base），证明miniconda安装完成了。

创建一个conda环境，激活进入该环境

代码块

```
1 conda create -n rknn-toolkitlite2 python=3.8
2 conda activate rknn-toolkitlite2
```

安装一些依赖

代码块

```
1 pip install opencv-python -i https://pypi.tuna.tsinghua.edu.cn/simple
```

2.进入rknn-toolkit-lite2/packages下，把下面标记的文件拷贝到开发板的ubuntu20系统下

```
miniconda3 Miniconda3-latest-Linux-aarch64.sh packages
(rknn-toolkitlite2) root@topeet:~# cd packages/
(rknn-toolkitlite2) root@topeet:~/packages# ls
packages.md5sum
rknn_toolkit_lite2-2.3.2-cp310-cp310-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
rknn_toolkit_lite2-2.3.2-cp311-cp311-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
rknn_toolkit_lite2-2.3.2-cp312-cp312-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
rknn_toolkit_lite2-2.3.2-cp37-cp37m-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
rknn_toolkit_lite2-2.3.2-cp38-cp38-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
rknn_toolkit_lite2-2.3.2-cp39-cp39-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
(rknn-toolkitlite2) root@topeet:~/packages#
```

3.执行命令进行安装

代码块

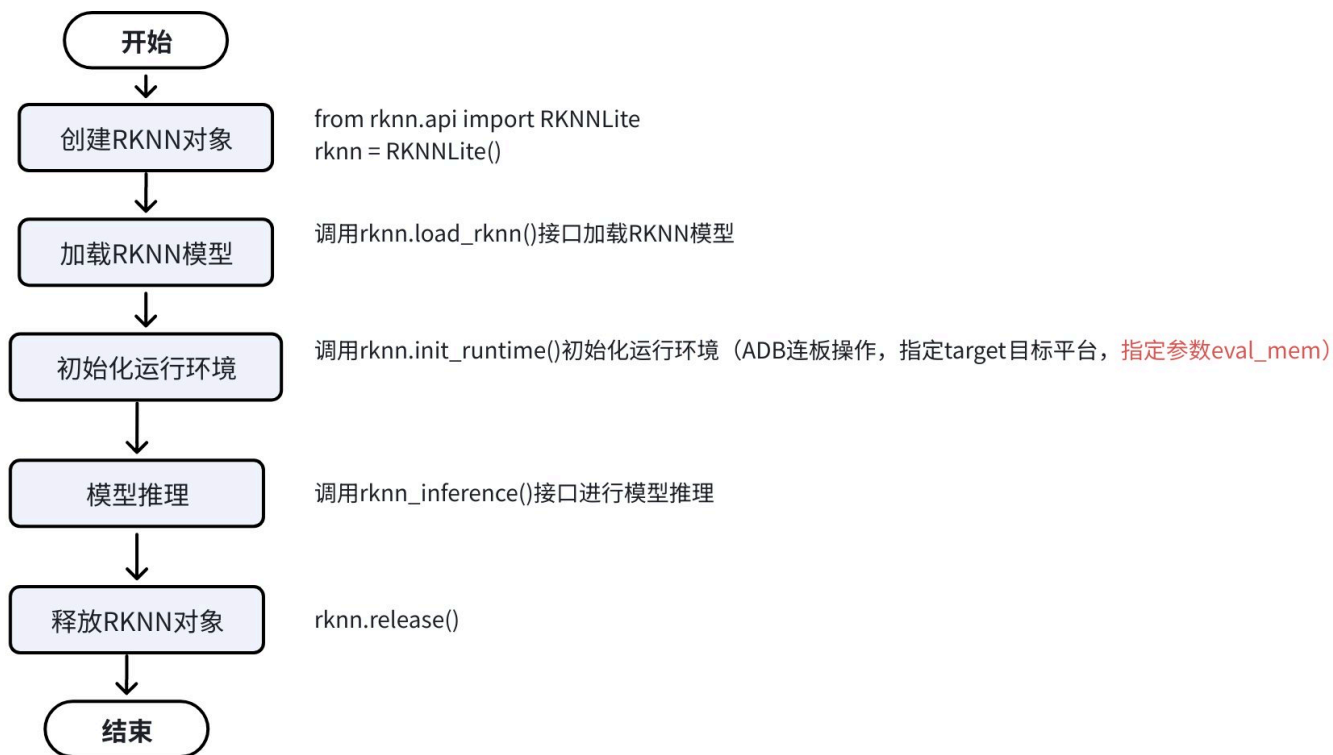
```
1 pip install rknn_toolkit_lite2-2.3.2-cp38-cp38-
  manylinux_2_17_aarch64.manylinux2014_aarch64.whl -i
  https://pypi.tuna.tsinghua.edu.cn/simple
```

4.进行测试

代码块

```
1 from rknnlite.api import RKNNLite
2 RKNNLite()
```

3. 使用流程



注:

- 1.在调用 inference 接口进行推理之前，需要获取输入数据，并做相应的预处理，然后根据输入信息设置inference 接口中的各项参数。
- 2.在调用 inference 接口后，通常需要对推理结果进行相应的处理，以完成应用程序相关功能。

4. API介绍

4.1 RKNNLite初始化及对象释放

在使用RKNN-Toolkit Lite2时，需要先调用 `RKNNLite()` 方法初始化一个 `RKNNLite` 对象，并在用完后调用该对象的 `rknn_lite.release()` 方法将资源释放掉。

初始化 `RKNNLite` 对象时，可以设置 `verbose` 和 `verbose_file` 参数，以打印详细的日志信息。其中 `verbose` 参数指定是否要在屏幕上打印详细日志信息；如果设置了 `verbose_file` 参数，且 `verbose` 参数值为 `True`，日志信息还将写到这个参数指定的文件中。

举例如下：

代码块

```
1  # 将详细的日志信息输出到屏幕，并写到inference.log文件中
2  rknn_lite = RKNNLite(verbose=True, verbose_file='./inference.log')
3  # 只在屏幕打印详细的日志信息
4  rknn_lite = RKNNLite(verbose=True)
5  ...
```

4.2 加载RKNN模型

API	load_rknn
描述	加载RKNN模型
参数	path: RKNN模型路径
返回值	0: 加载成功; -1: 加载失败。

举例如下：

代码块

```
1 # 从当前目录加载resnet_18.rknn模型
2 ret = rknn_lite.load_rknn('./resnet_18.rknn')
```

4.3 初始化运行时环境

API	init_runtime
描述	初始化运行时环境。
参数	core_mask: NPU工作核心配置模式。可选值如下： RKNNLite.NPU_CORE_AUTO: 自动调度模式，模型将以单核模式自动运行在当前空闲的NPU核上。 RKNNLite.NPU_CORE_0: 模型运行在NPU Core0上。 RKNNLite.NPU_CORE_1: 模型运行在NPU Core1上。 RKNNLite.NPU_CORE_2: 模型运行在NPU Core2上。 RKNNLite.NPU_CORE_0_1: 模型同时运行在NPU Core0和NPU Core1上。 RKNNLite.NPU_CORE_0_1_2: 模型同时运行在NPU Core0，NPU Core1和NPU Core2上。 RKNNLite.NPU_CORE_ALL: 模型同时运行在所有NPU核上。 默认值为 <code>NPU_CORE_AUTO</code>，即默认使用的是自动调度模式。 注：该参数仅对RK3576 / RK3588有效。
返回值	0: 初始化运行时环境成功; -1: 初始化运行时环境失败。

举例如下：

代码块 初始化运行时环境

```
2 ret = rknn_lite.init_runtime(core_mask=RKNNLite.NPU_CORE_AUTO)
3 if ret != 0:
4     print('Init runtime environment failed')
5     exit(ret)
```

4.4 模型推理

API	inference
描述	对指定输入进行推理，返回推理结果。
参数	inputs: 输入数据，如OpenCV读取的图片（如果输入是四维的，需要手动扩成4维）。类型是list，列表成员是ndarray。
	data_format: 数据排列方式，类型是list，对于每个输入可选值"nhwc", "nchw"，只对四维输入有效。默认值为None。如果不填写该参数，对于4维输入，数据buffer应按照NHWC排列，对于非4维输入，数据buffer应按照模型输入要求的格式排列。如果要填写该参数，对于非4维输入，数据buffer应按照模型输入要求的格式排列（不管填"nhwc"还是"nchw"）；对于4维输入，数据buffer应按照该参数设置的格式排列；对于多输入模型，填写该参数时要包括所有输入。
API	inference
返回值	results: 推理结果，类型是list，列表成员是ndarray。

以分类模型为例，模型推理代码参考如下：

代码块

```
1 # Get input data
2 img = cv2.imread('./space_shuttle_224.jpg')
3 img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
4 img = np.expand_dims(img, 0)
5 # Inference
6 outputs = rknn_lite.inference(inputs=[img])
7 # Show the classification results
8 show_top5(outputs)
```

4.5 查询SDK版本

API	get_sdk_version
描述	获取Runtime， 驱动和RKNN模型版本信息。注：使用该接口前必须完成模型加载和初始化运行环境。
参数	无
返回值	sdk_version: runtime， 驱动版本信息。类型为字符串。

举例如下：

代码块

```
1  # 获取SDK版本信息
2  sdk_version = rknn_lite.get_sdk_version()
```

返回的SDK信息参考如下：

代码块

```
1  =====
2  RKNN VERSION:
3  API: 1.5.2 (71720f3fc@2023-08-21T09:29:52)
4  DRV: 0.7.2
5  =====
```

4.6 查询模型可运行平台

API	list_support_target_platform
描述	查询给定RKNN模型可运行的芯片平台。
API	list_support_target_platform
参数	rknn_model: RKNN模型路径。如果不指定模型路径，则按类别打印RKNN-Toolkit Lite2当前支持的芯片平台。
返回值	support_target_platform: 返回模型可运行的芯片平台。如果RKNN模型路径为空或不存在，返回None.

5. 代码测试

代码块

```
1  from rknnlite.api import RKNNLite
2  import cv2
3  import numpy as np
4
5  def show_outputs(output):
6      output_sorted = sorted(output, reverse=True)
7      top5_str = '\n-----TOP 5-----\n'
8      for i in range(5):
9          value = output_sorted[i]
10         index = np.where(output == value)
11         for j in range(len(index)):
12             if (i + j) >= 5:
13                 break
14             if value > 0:
15                 topi = '{}: {}\n'.format(index[j], value)
16             else:
17                 topi = '-1: 0.0\n'
18             top5_str += topi
19     print(top5_str)
20
21 def show_perfs(perfs):
22     perfs = 'perfs: {}\n'.format(perfs)
23     print(perfs)
24
25 def softmax(x):
26     return np.exp(x)/sum(np.exp(x))
27
28 if __name__ == '__main__':
29     rknn = RKNNLite()
30
31     # 使用load_rknn接口直接加载RKNN模型
32     rknn.load_rknn(path="./resnet18.rknn")
33
34     # 调用init_runtime接口初始化运行时环境
35     rknn.init_runtime(
36         core_mask=RKNNLite.NPU_CORE_AUTO, # 表示NPU的调度模式
37     )
38
39     # 使用opencv获取推理的图片数据
40     img = cv2.imread(filename="./space_shuttle_224.jpg")
41     img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
42     img = np.expand_dims(img, 0)
43
44     #调用inference接口进行推理测试
45     outputs = rknn.inference(
```

```

46         inputs=[img],
47     )
48
49     show_outputs(softmax(np.array(outputs[0][0])))
50
51     rknn.release()

```

查看结果

```

(rknn-toolkitlite2) root@topeet:/userdata/15_rknn-toolkit-lite2-test# python resnet18.py
W rknn-toolkit-lite2 version: 2.3.2
I RKNN: [12:36:54.453] RKNN Runtime Information, librknrt version: 2.3.2 (429f97ae6b@2025-04-09T09:09:27)
I RKNN: [12:36:54.453] RKNN Driver Information, version: 0.9.8
I RKNN: [12:36:54.453] RKNN Model Information, version: 6, toolkit version: 2.3.2(compiler version: 2.3.2 (e045de
-04-07T19:48:25)), target: RKNPU v2, target platform: rk3588, framework name: PyTorch, framework layout: NCHW, mo
ence type: static_shape
W RKNN: [12:36:54.471] query RKNN_QUERY_INPUT_DYNAMIC_RANGE error, rknn model is static shape type, please export
n dynamic_shapes
W Query dynamic range failed. Ret code: RKNN_ERR_MODEL_INVALID. (If it is a static shape RKNN model, please ignor
ve warning message.)

-----TOP 5-----
[812]: 0.9996797442436218
[404]: 0.00024927116464823484
[657]: 1.2877121662313584e-05
[466]: 9.023943675856572e-06
[833 895]: 8.015345883904956e-06

(rknn-toolkitlite2) root@topeet:/userdata/15_rknn-toolkit-lite2-test#

```